Exercise 1: Let C1 and C2 be two consistent cuts. Show that the intersection of C1 and C2 is a consistent cut.

def A cut C is consistent iff: $e \longrightarrow e' \wedge e' \in C \Rightarrow e \in C$

Let $e' \in C = C_1 \cap C_2$ and e t.c. $e \longrightarrow e'$. By definition, since $e' \in C_1 \cap C_2$ we also have $e' \in C_1$ and $e' \in C_2$.

Howevevre, since $e \longrightarrow e'$ and C_1 is a consistent cut, e also HUST be in C_1.
Analog reasoning for C_2. Because $e \longrightarrow e'$ and C_2 is consistent, $e \in C_2$.
At this point, $e \in C_1 \wedge e \in C_2 \Rightarrow e \in C = C_1 \cap C_2$

So, $\forall e, e' \quad e' \in C = C_1 \cap C_2 \wedge e \longrightarrow e' \Rightarrow e \in C_1 \cap C_2 \Rightarrow C_1 \cap C_2$ CONSISTENT

Exercise 2: Let C1 and C2 be two consistent cuts. Show that the union of C1 and C2 is a consistent cut.

def A cut C is CONSISTENT iff: $e' \in C \wedge e \longrightarrow e' \Rightarrow e \in C$

Let $e' \in C = C_1 \cup C_2$ and e t.c. $e \longrightarrow e'$.

By definition of "consistent cut", since $e' \in C_1$ and $e \longrightarrow e'$, we have that also $e \in C_1$.
And $e \in C_1 \Rightarrow e \in C = C_1 \cup C_2$. (Same for C_2)

So, $\forall e' \in C = C_1 \cup C_2 \wedge e \longrightarrow e' \Rightarrow e \in C = C_1 \cup C_2 \Rightarrow C_1 \cup C_2$ CONSISTENT

Exercise 3: Show that every consistent global state can be reached by some consistent run.

Let E be the set of all events in the system. Exploiting the "happened before" relationship ($\longrightarrow$) we can create a sorting algorithm $sort(\cdot)$ such that:

- if $e \longrightarrow e'$ then e is placed before e'

Then we can apply this sorting algorithm to C and $E \setminus C$, in sequence, to create a run R:

$R = sort(C)$ followed by $sort(E \setminus C)$

At this point, R is also consistent because $\forall e, e' \in E$ t.c. $e \longrightarrow e'$:

- if $e \in C$ and $e' \in C$ then e will be placed before e' (by definition of $sort(\cdot)$)
- if $e \in C$ and $e' \in E \setminus C$ then e will be placed before e', since e will be in $sort(C)$ and e' will be in $sort(E \setminus C)$
- if $e \in E \setminus C$ and $e' \in C$ then C would not be consistent. So, this case is not possible
- if $e \in E \setminus C$ and $e' \in E \setminus C$ then e will be placed before e' (by definition of $sort(\cdot)$)

Exercise 4: Let C1 and C2 be two consistent cuts. If C1 is a subset of C2, then C2 is reachable from C1. (There exists a consistent run that reaches C1 and then reaches C2.)

Let E be the set of all events in the system. Exploiting the "happened before" relationship ($\longrightarrow$) we can create a sorting algorithm $sort(\cdot)$ such that:

- if $e \longrightarrow e'$ then e is placed before e'

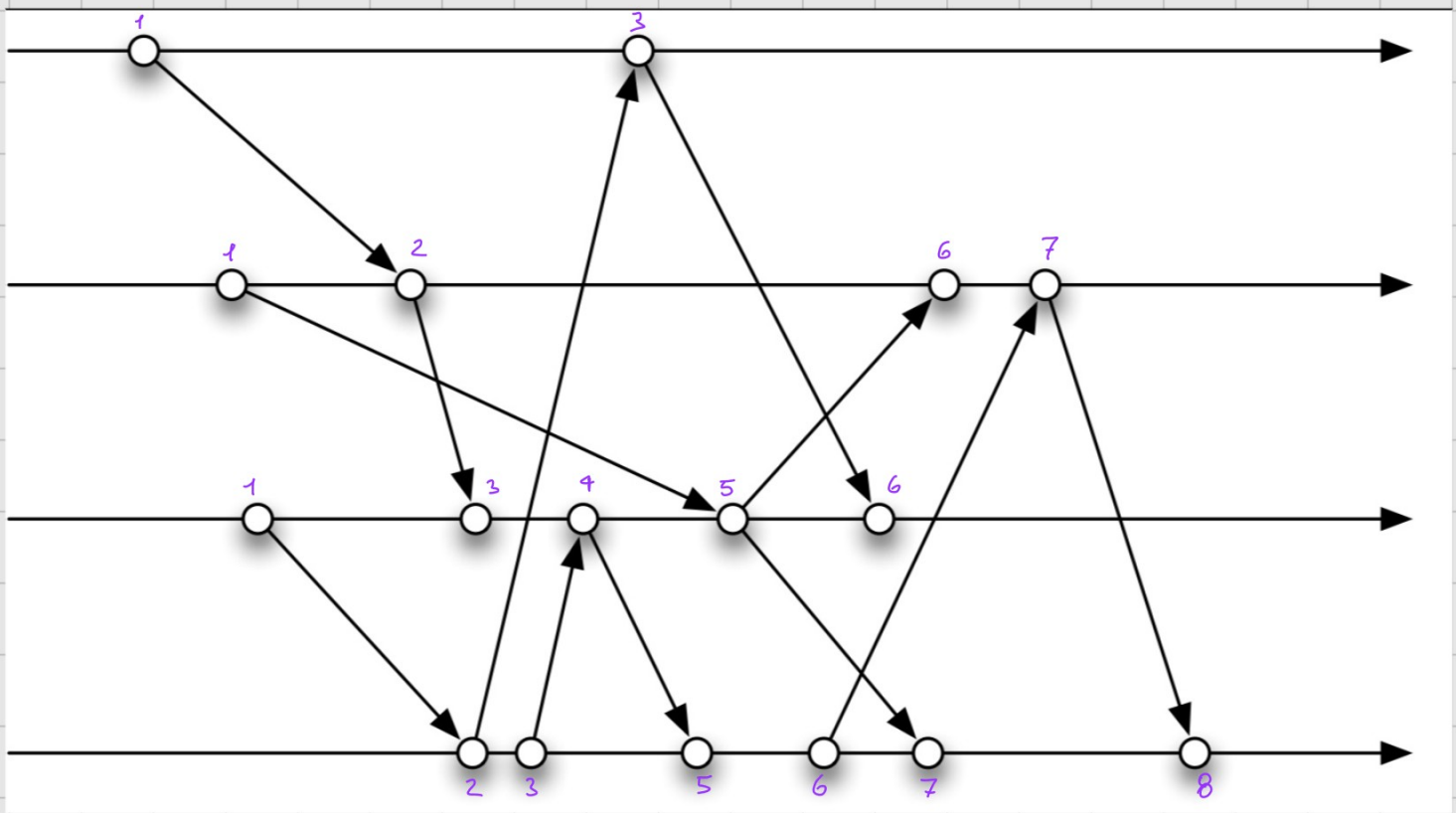
Since $C_1 \subseteq C_2$ we can divide E into three disjoint sets: C_1, $C_2 \setminus C_1$ and $E \setminus C_2$. Then we can apply this sorting algorithm to C_1, $C_2 \setminus C_1$ and $E \setminus C_2$, in sequence, to create a run R:

$$R = sort(C_1) \text{ followed by } sort(C_2 \setminus C_1) \text{ followed by } sort(E \setminus C_2)$$

At this point, R is also consistent because $\forall e, e' \in E$ t.c. $e \longrightarrow e'$:

- if e and e' are in the same set (both in C_1, both in $C_2 \setminus C_1$ or both in $E \setminus C_2$) then e will be placed before e' (by definition of $sort(\cdot)$)
- if e and e' are in different sets "left to right" ($e \in C_1 \wedge e' \in C_2 \setminus C_1 \cup E \setminus C_1$ or $e \in C_2 \setminus C_1 \wedge e' \in E \setminus C_2$) then e will be placed before e' by construction of R ($sort(C_1)$ appears before $sort(C_2 \setminus C_1)$ that appears before $sort(E \setminus C_2)$)
- if e and e' are in different sets "right to left", C_1 or C_2 would not be consistent. So this case is not possible.

Exercise 5: Label with proper logical clock all the events of the distributed computation in image vector.pdf. (You can consider events that receive a message and immediately send it as single events.)



Exercise 6: Show that the history of an event is a consistent cut.

def $H(e') = \{e : e \longrightarrow e'\}$

def A cut C is consistent if $\forall e, e' \; e \longrightarrow e' \wedge e' \in C \Rightarrow e \in C$

Let's proceed by CONTRADICTION. Let's assume that $H(e')$ is the history of the event e' and it is NOT a CONSISTENT CUT. This means that $\exists e : e \longrightarrow e'$ but $e \notin H(e')$.

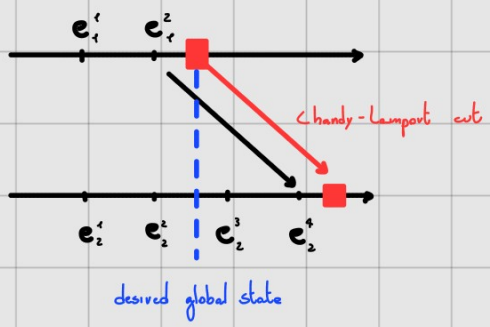
However, by definition of History, any event $e \longrightarrow e'$ MUST be in $H(e')$. So, in this case, $H(e')$ is not a valid history for $e' \Rightarrow$ CONTRADICTION.

Exercise 7: Show that the Chandy-Lamport Snapshot Protocol builds a consistent global state.

In this case it is sufficient to find a counter-example:

def The Global State is a "snapshot" of the entire system, constructed from the union of the local state of each process AT THE SAME TIME

So, basically, a global state is a straight cut on the time. With the Chandy-Lamport protocol, instead, we could get also cuts that are not global states.



Exercise 9: What good is a distributed snapshot when the system was never in the state represented by the distributed snapshot? Give an application of distributed snapshots.

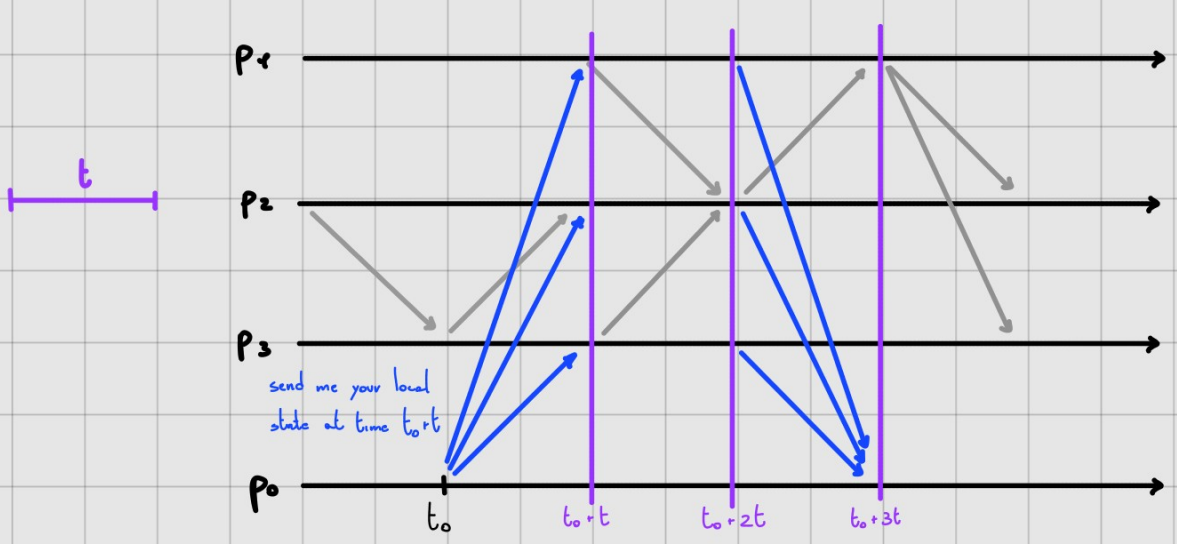
Exercise 10: Consider a distributed system where every node has its physical clock and all physical clocks are perfectly synchronized. Give an algorithm to record global state assuming the communication network is reliable. (Note that your algorithm should be simpler than the Chandy–Lamport algorithm.)

Let's assume that:

- t is the upper bound of time needed to deliver a message
- each message includes the exact timestamp in which it was sent

Protocol:

- At time t_0 a coordinator process p_0 broadcasts a message to all the other processes saying: "send me your local status at time $t_0 + t$".
- At time $t_0 + t$ all processes have received the message from p_0 and prepare their local state. However, since there could be messages that are still arriving, each process waits t time. If it receives a message with timestamp $< t_0 + t$ then it will be included in the local state.
- At time $t_0 + t + t$ we are sure that no other messages sent before $t_0 + t$ will arrive. So, each process finally sends it local state to p_0
- At time $t_0 + t + t$ p_0 will have received all the local states and can combine them to get the global state of the system at time $t_0 + t$



Exercise 11: What modifications should be done to the Chandy–Lamport snapshot protocol so that it records a strongly consistent snapshot (i.e., all channel states are recorded empty).

Each process p_i stops sending messages AS SOON AS it receives a "snapshot" message (marker). Then, p_i sends broadcasts its marker to all the other processes (as expected by the normal Chandy-Lamport Protocol).

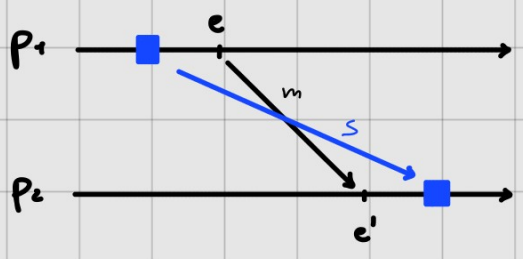
At this point, since the channel is FIFO and the marker is the last message sent by p_i, any other process p_j that receives that marker can be sure there are no other messages coming from p_i.

Applying this reasoning to all the processes, when a process receives all the markers from the others, it can be sure there are no messages in transit. So, it can record its local state. By combining all the local states we obtain the strongly consistent snapshot.

Exercise 12: Show that, if channels are not FIFO, then Chandy–Lamport snapshot algorithm does not work.

If the channel is NOT FIFO something like this could happen:

- the process p_1 takes the snapshot of its local state. Then sends the "snapshot request" s to the process p_2
- Immediatly after p_1 send a message m to p_2 (let's call this event e)
- Since the channel is NOT FIFO the process p_2 receives the message m before the marker s (let's call this event e').
- Consequently, the event e is NOT in the snapshot of p_1 but the event e' is in the snapshot of p_2. But $e \longrightarrow e'$... So, the final cut will not be consistent



Exercise 13: Let S0 be the global state when the Chandy-Lamport snapshot protocol starts, S be the global state built by the protocol, and S1 be the global state when the protocol ends. Show that S is reachable from S0 and that S1 is reachable from S. Remember that S might not have happened. (In other words, there is a consistent run that goes through S0, S1 and S.)

Due to the passage of time in the system we can obviously assume that $S_0 \subseteq S \subseteq S_f$.

At this point, the exercise becomes identical to Exercise4

Let E be the set of all events in the system. Exploiting the "happened before" relationship ($\longrightarrow$) we can create a sorting algorithm $sort(\cdot)$ such that:

- if $e \longrightarrow e'$ then e is placed before e'

And apply this sorting algorithm to S_0, $S \setminus S_0$ and $S \setminus S_f$, in sequence, to create a run R:

concatenate concatenate

$$R = sort(S_0) \mid sort(S_0 \setminus S) \mid sort(S \setminus S_f)$$

At this point, R is also consistent because $\forall e, e' \in E$ t.c. $e \longrightarrow e'$:

- if e and e' are in the same set (both in S_0, both in $S \setminus S_0$ or both in $S \setminus S_f$) then e will be placed before e' by definition of $sort(\cdot)$
- if e and e' are in different sets "left to right" ($e \in S_0 \wedge e' \in S_0 \setminus S \cup S \setminus S_f$ or $e \in S_0 \setminus S \wedge e' \in S \setminus S_f$) then e will be placed before e' by construction of R ($sort(C_f)$)
- if e and e' are in different sets "right to left", one of the sets would not be consistent. However, the Chandy-Lamport Protocol always generate consistent cuts... So this case is not possible